

Sistemas Operacionais

Simulador de Memória Virtual

Estudo prático sobre paginação e algoritmos de substituição baseados no livro Operating System Concepts.

Prof. Filipo Novo Mór

Objetivos do Trabalho —



Prática Teórica

Solidificar os conceitos de Memória Virtual e Paginação abordados no Capítulo 9 do Silberschatz.



Desenvolvimento

Implementar algoritmos de substituição de páginas em um ambiente simulado e controlado.

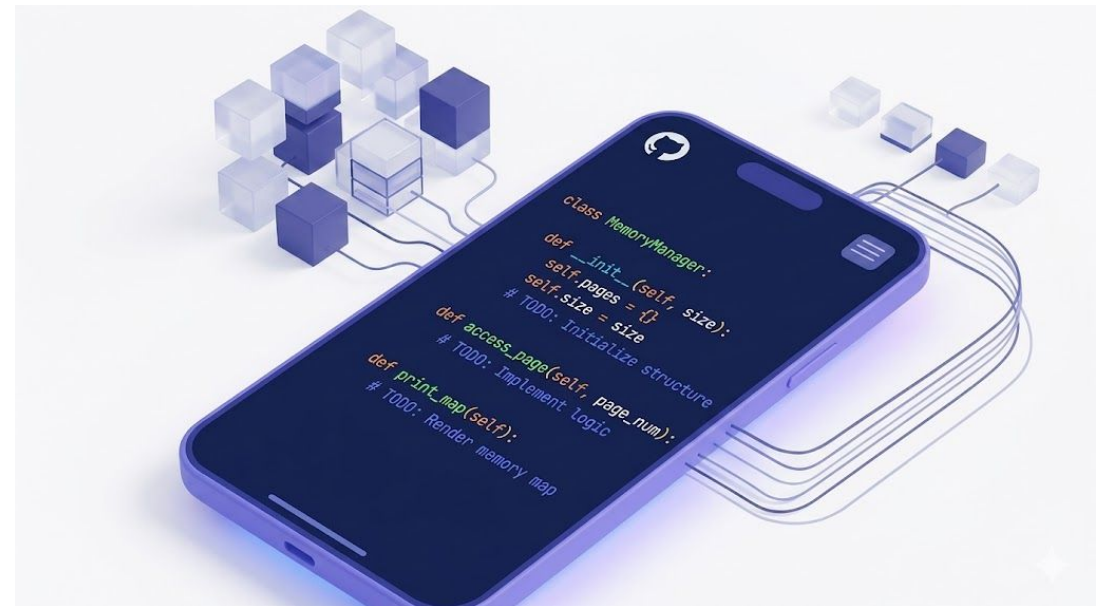


Análise de Dados

Contabilizar Page Faults e analisar a eficiência de diferentes lógicas para a mesma carga de trabalho.

Modelo de Código (Template)

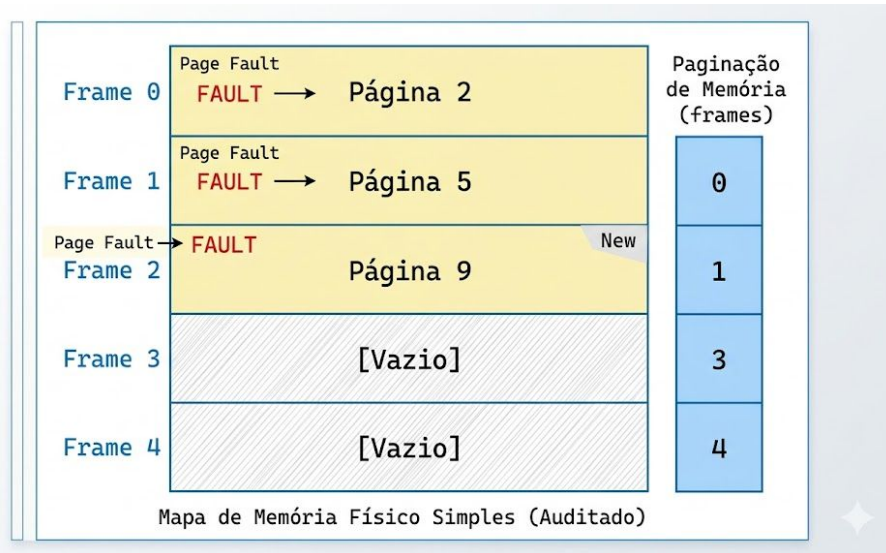
-  Repositório oficial: **ProfessorFilipo/MemSim**
-  Script base: `simulador_memoria.py`
-  Estrutura de leitura de arquivos e classes já pronta.
-  Foque apenas no TODO: Lógica e Mapa.



Requisitos Funcionais —

- **Mapear Acessos:** Converter referências do arquivo entrada.txt.
- ✖ **Detecção de Falhas:** Identificar se a página já reside nos frames da RAM simulada.
- ↻ **Substituição de Página:** Escolher a página "vítima" quando a memória estiver cheia.
- 📊 **Estatísticas:** Gerar relatório final com Total de Acessos, Faults e Taxa (%).

O Mapa de Memória



Saída no Terminal

A visualização passo a passo é obrigatória para a auditoria do algoritmo.

Identificação do Passo atual.

Estado de todos os Frames (Página X ou Vazio).

Sinalização clara do Frame alterado.

Status do acesso (HIT ou PAGE FAULT).

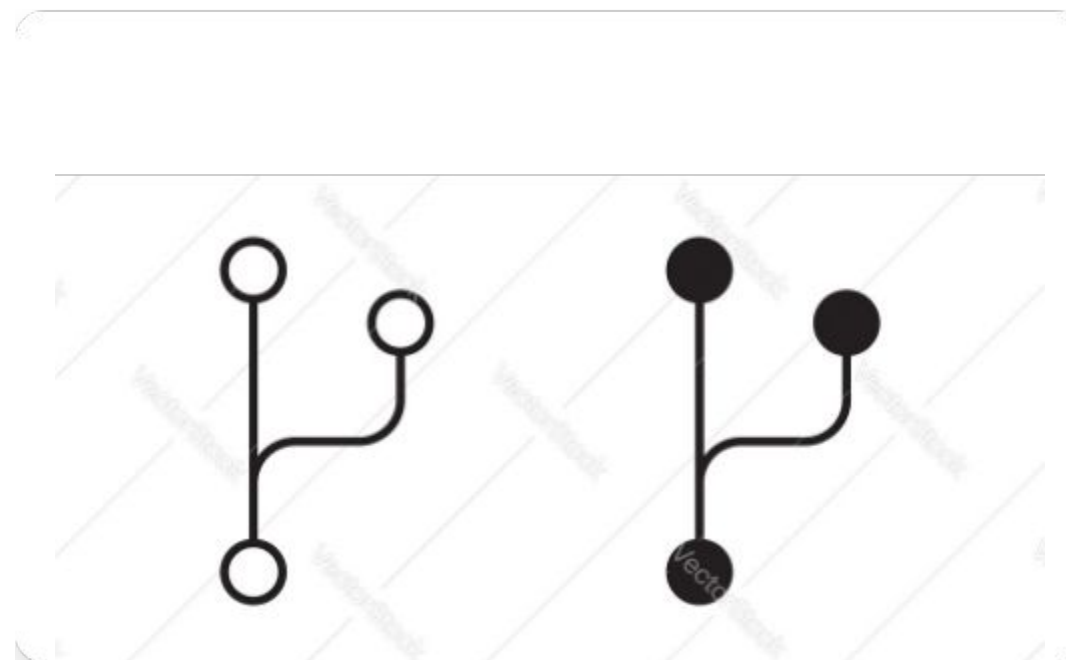
Distribuição dos Algoritmos

Cada um dos 15 grupos possui uma combinação única de dois algoritmos e quantidade de frames (3 ou 4).

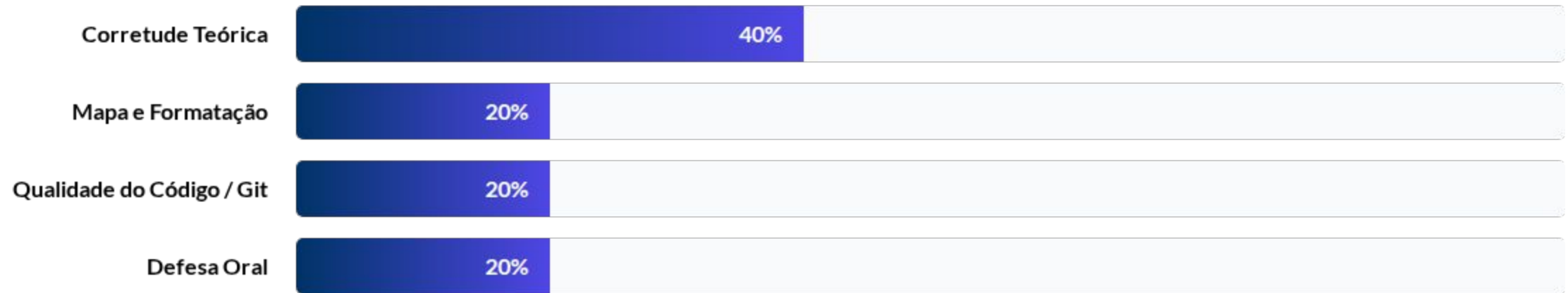
Lógica Base	Descrição Silberschatz	Requisitos de Dados
FIFO / LRU	Fila de entrada vs. Histórico de uso recente.	Lista ou Timestamps por Frame.
OPT (Ótimo)	Substitui a página que será usada mais tarde no futuro.	Look-ahead no arquivo de entrada.
Clock / Segunda Chance	Aproximação do LRU via bit de referência circular.	Bit de referência + Ponteiro.
LFU / MFU	Frequência de uso (Menos vs. Mais frequente).	Contador de acessos por Frame.

Entrega e Versionamento

- 🌐 **GitHub Público:** Link enviado até a data limite.
- 👥 **Auditoria de Commits:** Contribuições individuais serão verificadas.
- 👤 **Integrantes sem participação no histórico Git** receberão nota **ZERO**.
- ▶ **Execução Direta:** O professor clonará e rodará o código na hora.



Critérios de Avaliação —



Atenção: O código deve estar 100% funcional. Falhas na execução inicial invalidam os pontos de corretude.

Considerações Finais —

0
Bugs
Tolerados

15
Grupos Distintos

100%
Código Autoral

"O professor fará testes no dia da entrega com **novos arquivos de entrada** gerados aleatoriamente. Garanta que seu algoritmo trate qualquer sequência de números."

Dúvidas?

Bom trabalho a todos os grupos!

Prof. Filipo Novo Mór

filipo.mor@pucrs.br | github.com/ProfessorFilipo